Spaldotech Operations — Caddy Reverse Proxy (core-proxy-01) Build Guide

Oct 17, 2026 · @Aidan Spalding

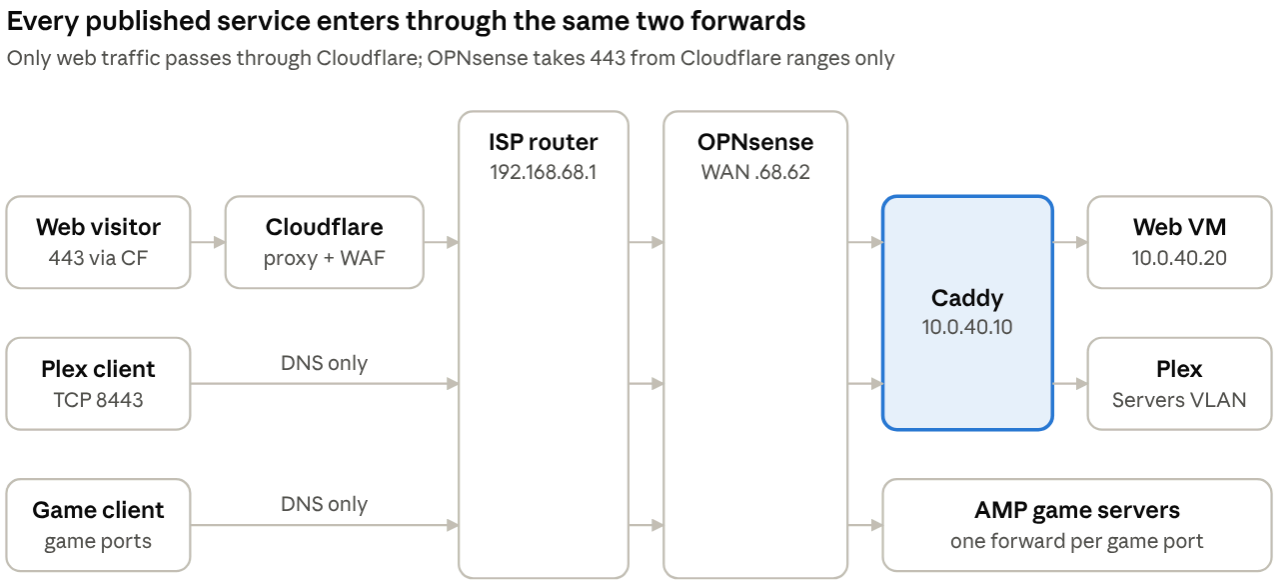
core-proxy-01 is a Debian 13 VM in the DMZ (VLAN 40, 10.0.40.10) running Caddy as the single HTTPS entry point for the lab. Websites under spaldotechoperations.co.uk arrive through Cloudflare's proxy on TCP 443, and Plex arrives directly on TCP 8443. Web, Plex and AMP's game ports all use the same two-hop port-forward pattern: ISP router, then OPNsense.

Field	Value
Document ID	SPO-HL-PRX-001
Version	1.0
Status	Draft — implementation baseline
Date	17 October 2026
Owner	Spaldotech Operations
Classification	Internal / Technical
Related standards	SPO-HL-SEC-001 (Security Hardening), Technical Implementation Guide §9, Naming Convention Standard §3.2

Scope: the core-proxy-01 VM, Caddy with the Cloudflare DNS and IP-range modules, Cloudflare zone and dynamic DNS settings, the unified port-forward register (web, Plex, AMP), and the OPNsense and host firewall rules that contain the proxy. Backend VMs are covered only where the proxy or a forward reaches them.

1. Purpose and design

All public services enter the lab the same way. The ISP router forwards a port to OPNsense's WAN (192.168.68.62), and OPNsense forwards it again to the service. Web and Plex land on Caddy in the DMZ. Game ports go straight to AMP, because they are not HTTP.



request path · visitor to backend

Web traffic passes through Cloudflare's proxy first, and OPNsense accepts TCP 443 only from Cloudflare's published address ranges. Plex and game traffic cannot use Cloudflare's proxy, so their hostnames are DNS-only and reach the home IP directly.

The design keeps the principles of both existing standards:

- **One register of forwards:** every exposed port is listed in §8.1 and nowhere else; adding a service means adding a row.
- **No exposed management:** Proxmox, OPNsense, Pi-hole, the AMP admin panel and PBS never get a forward or a public hostname.
- **Assume the DMZ is compromised:** OPNsense (§10) and nftables (§11) stop the proxy reaching private networks, apart from one narrow rule to Plex.
- **Plex stays off the Internet edge:** it sits behind Caddy, as Technical Implementation Guide §8.3 prefers, rather than having 32400 forwarded to the Plex host.

Trade-offs, stated explicitly as Technical Implementation Guide §9.4 asks:

- The home IP is public, because Plex and game hostnames must resolve to it. Cloudflare still adds its WAF and caching in front of the websites.
- Cloudflare decrypts visitor traffic to the websites at its edge. That is acceptable for public content.
- The design relies on the ISP router allowing port forwards. If the ISP moves the line behind carrier-grade NAT, forwards stop working and a tunnel becomes necessary.

2. Build parameters

The proxy follows the Naming Convention Standard (§3.2) and the .2–.9 infrastructure range of the DMZ. Adjust the VM ID and bridge name to match your host.

Parameter	Value
VM name / hostname	core-proxy-01
FQDN (inventory)	core-proxy-01.intraspald.tech
Friendly internal name	proxy.intraspald.tech (CNAME)
Proxmox VM ID	140 (suggested)
OS	Debian 13 (trixie), minimal, no desktop
vCPU / RAM / disk	2 vCPU / 2 GB / 16 GB (VirtIO SCSI, discard on)
Network	VirtIO NIC on the VLAN-aware internal bridge, VLAN tag 40
Address	10.0.40.10/24 (static)
Gateway	10.0.40.1 (OPNsense DMZ interface)
DNS resolvers	1.1.1.1, 1.0.0.1 (public; see §10)
Web backends	ops-web-01 10.0.40.20 (TCP 8080)
Plex backend	`<PLEX_IP>` on the Servers VLAN, TCP 32400
Public zone	spaldotechoperations.co.uk (Cloudflare DNS)
Backup job	bkp-core-proxy-daily

Port matrix

Service	Port	Bind / source	Direction
Caddy HTTPS (web)	TCP 443	10.0.40.10, from Cloudflare ranges only	Ingress via port forward
Caddy HTTPS (Plex)	TCP 8443	10.0.40.10, from any	Ingress via port forward
Caddy admin API	TCP 2019	127.0.0.1 only	Local
SSH	TCP 22	From admin sources only	Ingress

Service	Port	Bind / source	Direction
ACME / Cloudflare API / apt	TCP 443	Outbound to Internet	Egress
DNS	TCP/UDP 53	Outbound to 1.1.1.1, 1.0.0.1	Egress
NTP	UDP 123	Outbound to 10.0.40.1	Egress
Web backends	TCP 8080 (example)	To 10.0.40.20	Egress, intra-VLAN
Plex backend	TCP 32400	To `<PLEX_IP>`	Egress, via OPNsense

Caddy is the only listener the Internet can reach. Port 80 is never opened, because certificates come from the DNS-01 challenge. Replace `<PLEX_IP>` and `<AMP_IP>` throughout with the real addresses once those VMs are built.

3. Prerequisites

Complete these before creating the VM. Each one is a gate; do not continue past a failed item.

- [x] ~~spaldotechoperations.co.uk is an active zone in Cloudflare, with the registrar's nameservers pointing at Cloudflare.~~
- [x] ~~The ISP router (192.168.68.1) supports port forwarding, and the line has a real public IPv4 address rather than carrier-grade NAT: the router's WAN address matches what https://ifconfig.me reports.~~
- [x] ~~OPNsense's WAN address 192.168.68.62 is static or reserved on the ISP router, so forwards never point at a stale address.~~
- [x] ~~OPNsense has a DMZ interface on VLAN 40 with address 10.0.40.1/24, DHCP off or reserved outside .2–.9.~~
- [ ] The DMZ is recorded in SPO-HL-SEC-001. That standard currently lists 10.0.40.0/24 as Testing; update §2 and §8 so the tables match the built network.
- [ ] OPNsense configuration exported and Proxmox configuration backed up (SPO-HL-SEC-001 §3, step 1).
- [ ] Console access to Proxmox confirmed (iLO or local), so a lockout can be recovered.
- [ ] A Debian 13 netinst ISO is in Proxmox local storage, with its checksum verified.

Create the Cloudflare API token for certificates

Caddy uses this token only to write ACME `_acme-challenge` TXT records. A second, separate token for dynamic DNS is created in §8.3.

1. Cloudflare dashboard → My Profile → API Tokens → Create Token → Custom token.
2. Name: `core-proxy-01-acme-dns`.
3. Permissions: **Zone → Zone → Read** and **Zone → DNS → Edit**.
4. Zone resources: **Include → Specific zone → spaldotechoperations.co.uk**. Never "All zones".
5. Client IP address filtering: leave open (the home WAN IP may change) or set it to the current public IP if it is static.
6. TTL: set an end date 12 months out and add the rotation to the annual review (SPO-HL-SEC-001 §11).
7. Copy the token into the password manager. It is shown once and goes on the VM in §6.

The module documents a single token holding both permissions as the recommended approach; separate zone and DNS tokens are deprecated (caddy-dns/cloudflare).

4. Create the VM in Proxmox

Run from the Proxmox shell as the named admin. Replace `vmbr0`, `vm-storage` and the ISO filename with the names on your host.

```
qm create 140 \
  --name core-proxy-01 \
  --ostype l26 --machine q35 \
  --cores 2 --cpu x86-64-v2-AES --memory 2048 --balloon 0 \
  --scsihw virtio-scsi-single \
  --scsi0 vm-storage:16,discard=on,iothread=1,ssd=1 \
  --ide2 local:iso/debian-13-amd64-netinst.iso,media=cdrom \
  --boot 'order=scsi0;ide2' \
  --net0 virtio,bridge=vmbr0,tag=40,firewall=1 \
  --agent enabled=1 \
  --onboot 1 --startup order=3,up=30 \
  --tags 'core;dmz;proxy'
```

Startup order 3 places the proxy after OPNsense and Pi-hole, matching SPO-HL-SEC-001 §4.9. No USB, audio, serial or display passthrough is added (§4.8).

Proxmox guest firewall

The guest firewall is the inner layer around the VM. It only takes effect when the firewall is enabled at Datacenter level. Create `/etc/pve/firewall/140.fw`:

```
[OPTIONS]
enable: 1
policy_in: DROP
policy_out: ACCEPT
macfilter: 1

[IPSET admin_src]
10.0.10.0/24 # MGMT_NET
192.168.68.50 # admin workstation - replace with yours

[RULES]
IN HTTPS(ACCEPT) -log nolog # F1 websites; Cloudflare-only enforced by
OPNsense and nftables
IN ACCEPT -p tcp -dport 8443 -log nolog # F2 Plex via Caddy
IN SSH(ACCEPT) -source +admin_src -log nolog
IN Ping(ACCEPT) -source +admin_src -log nolog
```

Outbound stays open at this layer; OPNsense enforces egress in §10.

Install Debian 13

1. Start the VM and open the console.
2. Choose the text installer. Configure the network manually: 10.0.40.10/24, gateway 10.0.40.1, nameservers 1.1.1.1 1.0.0.1.
3. Hostname `core-proxy-01`, domain `intraspald.tech`.
4. Leave the root password blank. Debian then locks root and gives the first user sudo.
5. First user: `sysadmin`.
6. Partitioning: guided, entire disk, all files in one partition.
7. Software selection: tick only **SSH server** and **standard system utilities**.
8. Install GRUB to the disk, reboot, then detach the ISO: `qm set 140 --ide2 none,media=cdrom`.
9. Snapshot the clean install: `qm snapshot 140 base-install`.

5. Debian 13 base hardening

This applies the SPO-HL-SEC-001 §6 baseline to the proxy. The DMZ host is treated as the most exposed system in the lab, so nothing optional is installed.

5.1 Packages and updates

First switch apt to HTTPS, so updates keep working once the DMZ allows only TCP 443 outbound (§10). Edit whichever file the installer wrote, `/etc/apt/sources.list` or `/etc/apt/sources.list.d/debian.sources`:

```
sudo sed -i 's#http://deb.debian.org#https://deb.debian.org#g;
s#http://security.debian.org#https://security.debian.org#g' \
  /etc/apt/sources.list /etc/apt/sources.list.d/*.sources 2>/dev/null
grep -rh '^\(deb\|URIs\)' /etc/apt/sources.list /etc/apt/sources.list.d/
2>/dev/null   # all https
```

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y qemu-guest-agent unattended-upgrades apt-listchanges \
  nftables chrony curl gnupg ca-certificates debian-keyring debian-archive-
keyring
sudo systemctl enable --now qemu-guest-agent
sudo dpkg-reconfigure -plow unattended-upgrades
```

Do not install a compiler, Go toolchain, Docker or a desktop on this VM.

5.2 SSH keys and daemon

From the admin workstation, copy the key and confirm key login works before changing sshd:

```
ssh-copy-id sysadmin@10.0.40.10
ssh sysadmin@10.0.40.10   # must succeed without a password prompt
```

On the proxy:

```
sudo groupadd -f sshadmins
sudo usermod -aG sshadmins sysadmin
sudo tee /etc/ssh/sshd_config.d/99-hardening.conf >/dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
X11Forwarding no
AllowTcpForwarding no
AllowAgentForwarding no
MaxAuthTries 3
LoginGraceTime 30
AllowGroups sshadmins
EOF
sudo sshd -t && sudo systemctl reload ssh
```

Keep the current session open and test a second session before logging out. `AllowTcpForwarding no` stops the proxy being used as an SSH pivot into the DMZ.

5.3 Time

Point chrony at OPNsense so TLS and logs agree with the rest of the lab:

```
echo 'server 10.0.40.1 iburst' | sudo tee
/etc/chrony/sources.d/opnsense.sources
sudo sed -i 's/^pool /#pool /' /etc/chrony/chrony.conf
sudo systemctl restart chrony && chronyc sources
```

OPNsense NTP must listen on the DMZ interface for this to work.

5.4 Reduce the listening surface

```
sudo ss -lntup
systemctl list-unit-files --type=service --state=enabled
```

At this point only `sshd` and `chronyd` (loopback) should listen. Disable anything else that appears.

6. Install Caddy with the Cloudflare DNS module

The stock Caddy package has no DNS providers, so a custom build replaces the binary. It adds two modules: `github.com/caddy-dns/cloudflare` for DNS-01 certificates, and `github.com/WeidiDeng/caddy-cloudflare-ip`, which keeps Caddy's list of trusted

Cloudflare proxy addresses current. The apt package is still installed for its systemd unit, `caddy` user and directories. Caddy documents this as the supported pattern for custom builds on Debian (Caddy build docs).

6.1 Install the package

```
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' \
  | sudo gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-
keyring.gpg
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' \
  | sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo chmod o+r /usr/share/keyrings/caddy-stable-archive-keyring.gpg \
  /etc/apt/sources.list.d/caddy-stable.list
sudo apt update && sudo apt install -y caddy
sudo systemctl stop caddy
```

6.2 Fetch the custom build

Use Caddy's build server so no Go toolchain lands in the DMZ:

```
cd /tmp
curl -fL -o caddy \
  'https://caddyserver.com/api/download?
os=linux&arch=amd64&p=github.com%2Fcaddy-
dns%2Fcloudflare&p=github.com%2FWeidiDeng%2Fcaddy-cloudflare-ip'
chmod +x caddy
./caddy version
./caddy list-modules | grep -E
'dns.providers.cloudflare|http.ip_sources.cloudflare'   # both must print
```

Alternative: build on the admin workstation with `xcaddy build --with github.com/caddy-dns/cloudflare --with github.com/WeidiDeng/caddy-cloudflare-ip` (needs Go 1.26 or newer, newer than Debian 13's packaged Go) and copy the binary across with scp.

6.3 Swap the binary in

```
sudo dpkg-divert --divert /usr/bin/caddy.default --rename /usr/bin/caddy
sudo mv /tmp/caddy /usr/bin/caddy.custom
sudo update-alternatives --install /usr/bin/caddy caddy
/usr/bin/caddy.default 10
sudo update-alternatives --install /usr/bin/caddy caddy /usr/bin/caddy.custom
50
caddy list-modules | grep cloudflare
```

From here, `apt upgrade` only refreshes `caddy.default`. Upgrade the live binary with `sudo caddy upgrade`, which fetches the latest Caddy with the same plugins, then restart the service.

Before swapping, make sure nothing from an earlier manual install shadows the package. `ls /usr/local/bin/caddy` should find nothing, and `systemctl cat caddy | grep ExecStart` should show `/usr/bin/caddy`. A leftover binary in `/usr/local/bin`, or a hand-written unit in `/etc/systemd/system/caddy.service`, overrides the PATH and the service respectively. Move either aside (to `/root/`, say) before continuing.

6.4 Store the API token

```
sudo install -m 0640 -o root -g caddy /dev/null /etc/caddy/caddy.env
cloud
# add one line:
# CF_API_TOKEN=<token from §3>
```

The file is readable by root and the `caddy` group only. Never put the token in the Caddyfile, a shell history or a backup that is not encrypted.

6.5 systemd override

`sudo systemctl edit caddy` and add:

```
[Service]
EnvironmentFile=/etc/caddy/caddy.env
NoNewPrivileges=true
PrivateTmp=true
PrivateDevices=true
ProtectHome=true
ProtectSystem=strict
ReadWritePaths=/var/lib/caddy /var/log/caddy
ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
RestrictSUIDSGID=true
```

Then `sudo install -d -o caddy -g caddy -m 0750 /var/log/caddy` and `sudo systemctl daemon-reload`. Do not start Caddy until the Caddyfile in §7 is in place.

Never run Caddy with plain `sudo` against this configuration. Any file it creates in `/var/log/caddy` or `/var/lib/caddy` becomes root-owned, and the service, which runs as `caddy`, then exits at start-up. Validate as the `caddy` user, as shown in §7.1.

7. Caddyfile

Caddy binds to 10.0.40.10 and serves two kinds of site. Cloudflare-proxied websites listen on 443; DNS-only sites, currently just Plex, listen on 8443. It obtains real Let's Encrypt certificates for both through the Cloudflare DNS-01 challenge. It trusts the `Cf-Connecting-Ip` header only when the connection really comes from a Cloudflare address. Replace `/etc/caddy/Caddyfile` with the following, adjusting backends and the contact email.

```
{
        email hostmaster@spaldotechoperations.co.uk
        admin localhost:2019
        default_bind 10.0.40.10
        auto_https disable_redirects
        # First run only, to avoid Let's Encrypt rate limits; remove once
certificates issue:
        # acme_ca https://acme-staging-v02.api.letsencrypt.org/directory

        servers {
                trusted_proxies cloudflare {
                        interval 12h
                        timeout 15s
                }
                client_ip_headers Cf-Connecting-Ip
                protocols h1 h2
        }
}

(cf_tls) {
        tls {
                dns cloudflare {env.CF_API_TOKEN}
                resolvers 1.1.1.1 1.0.0.1
        }
}

(secure_headers) {
        header {
                Strict-Transport-Security "max-age=31536000;
includeSubDomains"
                X-Content-Type-Options "nosniff"
                X-Frame-Options "DENY"
                Referrer-Policy "strict-origin-when-cross-origin"
                Permissions-Policy "camera=(), microphone=(), geolocation=()"
                -Server
                -X-Powered-By
        }
```

```
}

(access_log) {
        log {
                output file /var/log/caddy/access.log {
                        roll_size 50MiB
                        roll_keep 14
                }
                format json
        }
}

# ---- Cloudflare-proxied sites (443, Cloudflare sources only) ----

spaldotechoperations.co.uk {
        import cf_tls
        import access_log
        redir https://www.spaldotechoperations.co.uk{uri} permanent
}

www.spaldotechoperations.co.uk {
        import cf_tls
        import secure_headers
        import access_log

        reverse_proxy 10.0.40.20:8080 {
                # Keep the two health_ lines only if the app answers 200 on
this path
                health_uri /healthz
                health_interval 15s
                header_up X-Real-IP {client_ip}
                header_down -Via
        }
}

# ---- DNS-only sites (8443, any source) ----

plex.spaldotechoperations.co.uk:8443 {
        import cf_tls
        import access_log
        header {
                Strict-Transport-Security "max-age=31536000"
                -Server
        }

        reverse_proxy <PLEX_IP>:32400 {
```

```
                flush_interval -1
                header_up X-Real-IP {client_ip}
        }
}
```

Notes on the choices:

- `default_bind 10.0.40.10` binds every site listener to the DMZ address only; the admin API stays on localhost. If the VM's address ever changes, update this line too, or Caddy will refuse to start.
- `trusted_proxies cloudflare` refreshes Cloudflare's published ranges every 12 hours. `{client_ip}` and the access log then show the real visitor on 443. On 8443, Plex clients connect directly, are not Cloudflare addresses, and so any forged `Cf-Connecting-Ip` header they send is ignored.
- `protocols h1 h2` stops Caddy advertising HTTP/3, which needs UDP. Only TCP is forwarded, so clients would otherwise try UDP first and then fall back.
- There is deliberately no global `log` block. Caddy's own messages, including start-up errors, go to the journal (`journalctl -u caddy`); only the per-site access logs are written to `/var/log/caddy` .
- Caddy already defaults to TLS 1.2 minimum with modern ciphers, which meets Technical Implementation Guide §9.3.
- `health_uri` takes a backend out of rotation when its check fails. With a single backend, that turns every request into an instant 503. Keep it only if the app returns 200 on `/healthz` , or point it at `/` .
- `header_down -Via` removes the `Via: 1.1 Caddy` header. `flush_interval -1` stops Caddy buffering Plex streams, and WebSocket upgrades for the Plex web app are proxied automatically.
- Add a Content-Security-Policy per site once the application's needs are known.
- Adding a site: a Cloudflare-proxied site copies the `www` block; a DNS-only site copies the `plex` block on `:8443` . Then follow §13.2.

7.1 Validate and start

```
sudo caddy fmt --overwrite /etc/caddy/Caddyfile
# Validate as the caddy user, so no root-owned files appear in /var/log/caddy
or /var/lib/caddy
sudo -u caddy -H bash -c 'set -a; . /etc/caddy/caddy.env; /usr/bin/caddy
validate --config /etc/caddy/Caddyfile'
sudo systemctl enable --now caddy
journalctl -u caddy -f        # wait for "certificate obtained successfully"
sudo ss -lntp | grep caddy    # 10.0.40.10:443, 10.0.40.10:8443 and
127.0.0.1:2019 only
```

If issuance succeeded against staging, remove the `acme_ca` line, run `sudo systemctl reload caddy`, and confirm production certificates issue.

7.2 Troubleshooting

These are the failures met during the first build, in the order they appeared.

Symptom	Cause	Fix
`module not registered: http.ip_sources.cloudflare`	The binary being run lacks the module, often a leftover `/usr/local/bin/caddy` that shadows `/usr/bin/caddy` on the PATH	`readlink -f "$(command -v caddy)"` must show `/usr/bin/caddy.custom`. Move stray binaries aside and repeat §6.2–6.3
Service fails with `status=203/EXEC`	A hand-written `/etc/systemd/system/caddy.service` replaces the package unit and points at a binary that no longer exists	Move that file aside, run `systemctl daemon-reload`, and confirm `ExecStart=/usr/bin/caddy`
Service exits straight after "adapted config to JSON", with no error in the journal	Files in `/var/log/caddy` or `/var/lib/caddy` are root-owned after Caddy was run with plain sudo	`sudo chown -R caddy:caddy /var/log/caddy /var/lib/caddy`; validate as the `caddy` user (§7.1)
Every request returns 503 in under 1 ms, with size 0	Every backend is marked unhealthy, usually by `health_uri`	From the proxy, `curl` the backend and its health path; fix the app or remove `health_uri`
`curl` to the backend returns `Connection refused`	Nothing listens on that address and port on the backend	Run `ss -lntp` on the backend; bind the app to `0.0.0.0` or its DMZ address
`curl` to the backend times out	A drop rule: `BACKENDS` in the proxy's nftables, or the backend's own or Proxmox firewall	Check the proxy's kernel log for `nft-out-private`, then the backend's rules

8. Publishing: port forwards and dynamic DNS

Every exposed port in the lab is a row in the register below, forwarded twice: once on the ISP router to OPNsense's WAN, then once on OPNsense to the service. A forward that is not in this table must not exist on either device.

8.1 Forward register

#	Service	Public name	Cloudflare	Port	ISP router →	OPNsense →	Allowed sources
F1	Websites	spaldotechoperations.co.uk, www	Proxied	TCP 443	192.168.68.62:443	10.0.40.10:443	CF_IPV4 only
F2	Plex	plex.spaldotechoperations.co.uk	DNS only	TCP 8443	192.168.68.62:8443	10.0.40.10:8443	Any
F3	AMP instance (example: Minecraft)	play.spaldotechoperations.co.uk	DNS only	TCP 25565	192.168.68.62:25565	`<AMP_IP>`:25565	Any

AMP rows are added when an instance goes live and removed when it is retired, per Technical Implementation Guide §10.2. Never forward a range, the AMP admin port, Plex's 32400, or any management port.

8.2 ISP router (192.168.68.1)

1. Create one forward per register row: external port = internal port, target 192.168.68.62, protocol as listed.
2. Disable UPnP. Otherwise Plex, consoles and other devices open their own forwards outside the register.
3. Disable remote (WAN-side) management of the router itself.
4. Leave DMZ host / "expose host" off; it would send every unmatched port to OPNsense.

8.3 OPNsense forwards

In Firewall → NAT → Port Forward (named Destination NAT in newer releases), add one rule per register row:

Field	F1	F2	F3
Interface	WAN	WAN	WAN
Protocol	TCP	TCP	TCP (per game)
Source	CF_IPV4 (§10.1)	any	any
Destination	WAN address	WAN address	WAN address

Field	F1	F2	F3
Destination port	443	8443	25565
Redirect target	10.0.40.10:443	10.0.40.10:8443	`<AMP_IP>`:25565
Filter rule association	Add associated filter rule	Add associated filter rule	Add associated filter rule
Description	F1 web via Cloudflare	F2 Plex via Caddy	F3 AMP Minecraft

The source restriction on F1 matters: without it, anyone who learns the home IP could bypass Cloudflare's WAF and reach Caddy directly. Name each rule with its register number so the register, the router and OPNsense can be compared line by line.

8.4 Plex settings

In Plex Web → Settings, on the Plex server:

- **Remote Access:** disabled. Plex must not try to open or advertise 32400.
- **Network → Custom server access URLs:** `https://plex.spaldotechoperations.co.uk:8443`
- **Network → Secure connections:** Required.
- **Network → LAN Networks:** list the home and trusted lab subnets, but never 10.0.40.0/24. Otherwise every remote stream through Caddy is treated as local.
- **Network → List of IP addresses and networks that are allowed without auth:** leave empty, and never include 10.0.40.10.

8.5 Dynamic DNS

The home IP can change, so OPNsense keeps two Cloudflare records current. Create the records first (§9); the updater only changes their address.

1. Create a second Cloudflare API token, `opnsense-ddns`: Zone → Zone → Read and Zone → DNS → Edit on spaldotechoperations.co.uk only. Keep it separate from the Caddy token, so either can be revoked alone.
2. On OPNsense, install the `os-ddclient` plugin. Then go to Services → Dynamic DNS and add an account with service Cloudflare, the token as the password, and hostnames `spaldotechoperations.co.uk,home.spaldotechoperations.co.uk`.
3. Set the IP check method to an external web service, not the interface. OPNsense's WAN holds the private 192.168.68.62, so only an outside check sees the real public IP.
4. Confirm both records in Cloudflare match https://ifconfig.me from a home device.

8.6 Access from inside the home

Add Pi-hole local DNS records so home clients reach DNS-only services directly, without relying on NAT loopback at the ISP router:

Name	Answer
plex.spaldotechoperations.co.uk	10.0.40.10
play.spaldotechoperations.co.uk	`<AMP_IP>`

The certificate still matches, because Caddy serves the same hostname. The websites keep resolving to Cloudflare from inside, which works without any change.

9. Cloudflare zone settings

Set these in the Cloudflare dashboard for spaldotechoperations.co.uk. The DNS records come first, because dynamic DNS (§8.5) only updates existing records.

9.1 DNS records

Name	Type	Content	Proxy status	Kept current by
spaldotechoperations.co.uk	A	Home public IP	Proxied	OPNsense ddclient
www	CNAME	spaldotechoperations.co.uk	Proxied	Follows apex
home	A	Home public IP	DNS only	OPNsense ddclient
plex	CNAME	home.spaldotechoperations.co.uk	DNS only	Follows home
play	CNAME	home.spaldotechoperations.co.uk	DNS only	Follows home

Plex and game hostnames must stay DNS only. Cloudflare's proxy carries only HTTP(S) on standard ports, and its terms restrict serving large volumes of video through it.

9.2 Zone settings

Area	Setting	Value
SSL/TLS → Overview	Encryption mode	Full (strict); Caddy holds a valid Let's Encrypt certificate

Area	Setting	Value
SSL/TLS → Edge Certificates	Always Use HTTPS	On
SSL/TLS → Edge Certificates	Minimum TLS version	1.2
SSL/TLS → Edge Certificates	TLS 1.3	On
SSL/TLS → Edge Certificates	HSTS	Off here; Caddy sets it (§7), so it is defined once
DNS → Settings	DNSSEC	Enabled, DS record added at the registrar
DNS → Records	CAA	`0 issue "letsencrypt.org"` plus `0 iodef "mailto:hostmaster@spaldotechoperations.co.uk"`
Security → WAF	Managed rules	Cloudflare free managed ruleset on
Security → Bots	Bot Fight Mode	On (test the site afterwards)
Security → WAF → Rate limiting	One rule	e.g. 100 requests / 10 s per IP on paths that accept POST
Notifications	Origin errors	Alert on sustained 5xx / origin unreachable for the zone

Enabling CAA restricts which authorities may issue certificates for the zone. Check after saving that Cloudflare's own edge certificate is still active, since Universal SSL uses its own authorities.

What must never be published

Proxmox, OPNsense, Pi-hole, the AMP admin panel, PBS and iLO never get a forward in §8.1 or a public DNS record. Remote administration stays on WireGuard, as both existing standards require. A non-public web app published later (an internal dashboard, say) goes behind a Cloudflare Access policy on a proxied hostname before its forward row is added.

10. OPNsense rules for the DMZ (VLAN 40)

The DMZ may reach only the Internet services the proxy needs, plus one narrow path to Plex, and nothing else private. One block on all RFC 1918 space enforces SPO-HL-SEC-001 §8 (DMZ → Home and DMZ → Management denied). Inbound, the WAN accepts only the

register in §8.1.

10.1 Aliases (Firewall → Aliases)

Alias	Type	Content
CF_IPV4	URL Table (IPs), refresh 1 day	https://www.cloudflare.com/ips-v4
DMZ_NET	Network	10.0.40.0/24
DMZ_PROXY	Host	10.0.40.10
DMZ_WEB	Host	10.0.40.20
PLEX_HOST	Host	`<PLEX_IP>`
AMP_HOST	Host	`<AMP_IP>`
RFC1918	Network	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
HOME_NET	Network	192.168.68.0/24
CF_DNS	Host	1.1.1.1, 1.0.0.1
ADMIN_SRC	Network	10.0.10.0/24, plus the admin workstation's 192.168.68.x/32

After saving CF_IPV4, check Firewall → Diagnostics → Aliases shows it populated (around 15 prefixes) before relying on F1.

10.2 DMZ interface rules (evaluated top-down)

#	Action	Proto	Source	Destination	Port	Purpose
1	Pass	UDP	DMZ_NET	This Firewall	123	NTP from OPNsense
2	Pass	TCP/UDP	DMZ_PROXY	CF_DNS	53	Public DNS for ACME
3	Block + log	TCP/UDP	DMZ_NET	any	53	No other resolvers
4	Block + log	any	DMZ_NET	This Firewall	any	No access to the firewall GUI or SSH
5	Pass	TCP	DMZ_PROXY	PLEX_HOST	32400	Caddy to Plex: the only DMZ → private exception
6	Block + log	any	DMZ_NET	RFC1918	any	No route to Home, MGMT, Servers or Storage

#	Action	Proto	Source	Destination	Port	Purpose
7	Pass	TCP	DMZ_PROXY	any	443	ACME, Cloudflare API and IP list, apt, Caddy downloads
8	Pass	TCP	DMZ_WEB	any	443	Backend OS updates only
9	Block + log	any	DMZ_NET	any	any	Default deny, logged

Because rule 6 precedes the pass rules, `any` in rules 7–8 can only ever match public addresses. Rule 5 is the exception Technical Implementation Guide §8.3 allows: one source, one host, one port. Also restrict 32400 on the Plex host's own firewall to 10.0.40.10 and the home and trusted subnets. No TCP 80 rule is needed because §5.1 switches apt to HTTPS.

10.3 Inbound rules

Interface	Action	Source	Destination	Port	Purpose
WAN	Pass (from §8.3)	CF_IPV4	10.0.40.10	TCP 443	F1 websites
WAN	Pass (from §8.3)	any	10.0.40.10	TCP 8443	F2 Plex
WAN	Pass (from §8.3)	any	AMP_HOST	per game	F3+ AMP instances
WAN (home side, routed)	Pass	HOME_NET	DMZ_PROXY	TCP 8443	Plex from home via split DNS (§8.6)
WAN (home side, routed)	Pass	HOME_NET	AMP_HOST	per game	Games from home via split DNS
WAN (home side, routed)	Pass	admin workstation /32	DMZ_PROXY	TCP 22	Admin SSH
MGMT	Pass	MGMT_NET	DMZ_PROXY	TCP 22	Admin SSH

The three WAN pass rules are the associated rules OPNsense creates with each port forward; keep them linked, so deleting a forward also deletes its rule. Trusted VLANs that should reach Plex or games get the same two rules on their own interfaces. Nothing else may reach VLAN 40.

10.4 Points that OPNsense cannot enforce

- Proxy-to-web-backend traffic stays inside VLAN 40 and never crosses OPNsense. Restrict it on each backend: allow its application port from 10.0.40.10 only, in both the backend's nftables and its Proxmox guest firewall.

- Confirm outbound NAT covers 10.0.40.0/24 (Firewall → NAT → Outbound). Automatic mode includes it once the interface is assigned.
- Leave NAT reflection off. Split DNS (§8.6) gives home clients a direct path instead.
- Run Suricata in IPS mode on WAN, per Technical Implementation Guide §5.4. It now inspects every forwarded connection, including the plaintext game protocols.

11. Host firewall (nftables)

The proxy accepts 443 only from Cloudflare, 8443 from anyone, and SSH from admin sources. Its egress mirrors the OPNsense rules in §10, so a compromise of Caddy meets two independent filters. Write `/etc/nftables.conf`, replacing the workstation and Plex addresses:

```
#!/usr/sbin/nft -f
flush ruleset

define ADMIN_SRC = { 10.0.10.0/24, 192.168.68.50 }
define CF_DNS    = { 1.1.1.1, 1.0.0.1 }
define BACKENDS  = { 10.0.40.20 }
define PLEX      = <PLEX_IP>
define PRIVATE   = { 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16 }

table inet filter {
  # Filled by cf-ips.service (11.2); empty until it first runs
  set cf4 {
    type ipv4_addr
    flags interval
  }

  chain input {
    type filter hook input priority filter; policy drop;
    ct state established,related accept
    ct state invalid drop
    iif "lo" accept
    ip saddr $ADMIN_SRC icmp type echo-request limit rate 5/second accept
    icmp type { destination-unreachable, time-exceeded } accept
    tcp dport 443 ip saddr @cf4 accept
    tcp dport 8443 accept
    ip saddr $ADMIN_SRC tcp dport 22 ct state new limit rate 10/minute accept
    limit rate 5/minute log prefix "nft-in-drop: " level info
  }

  chain forward {
    type filter hook forward priority filter; policy drop;
```

```
  }

  chain output {
    type filter hook output priority filter; policy drop;
    ct state established,related accept
    oif "lo" accept
    meta l4proto icmp accept
    ip daddr 10.0.40.1 udp dport 123 accept
    ip daddr $CF_DNS meta l4proto { tcp, udp } th dport 53 accept
    ip daddr $BACKENDS tcp dport 8080 accept
    ip daddr $PLEX tcp dport 32400 accept
    ip daddr $PRIVATE limit rate 5/minute log prefix "nft-out-private: "
level warn
    ip daddr $PRIVATE drop
    tcp dport 443 accept
    limit rate 5/minute log prefix "nft-out-drop: " level info
  }
}
```

Replies to forwarded connections leave through `ct state established,related`, so no output rule is needed for them.

11.1 Apply with a dead-man switch

Keep the current SSH session open throughout.

```
sudo nft -c -f /etc/nftables.conf                # syntax check only
sudo systemd-run --unit=nft-rollback --on-active=5m /usr/sbin/nft flush
ruleset
sudo nft -f /etc/nftables.conf
# from a SECOND terminal: ssh sysadmin@10.0.40.10 must still work
sudo systemctl stop nft-rollback.timer           # cancel the rollback
sudo systemctl enable --now nftables
sudo nft list ruleset
```

If the second session fails, wait five minutes: the timer flushes the ruleset and access returns. After a day of running, check `journalctl -k | grep nft-` for anything legitimate being dropped.

11.2 Keep the Cloudflare set current

A small service loads Cloudflare's published ranges into `cf4` at boot, whenever nftables restarts, and daily. It refuses to load a list that looks wrong, leaving the previous contents in place.

Create `/usr/local/sbin/update-cf-ips` (mode 0755):

```
#!/bin/sh
set -eu
list=$(curl -fsS --max-time 20 https://www.cloudflare.com/ips-v4)
if echo "$list" | grep -Evq '^[0-9]{1,3}(\.[0-9]{1,3}){3}/[0-9]{1,2}$'; then
  echo "unexpected content in Cloudflare IP list" >&2; exit 1
fi
count=$(echo "$list" | grep -c .)
[ "$count" -ge 5 ] || { echo "only $count prefixes, refusing" >&2; exit 1; }
elems=$(echo "$list" | paste -sd, -)
nft -f - <<EOF
flush set inet filter cf4
add element inet filter cf4 { $elems }
EOF
logger -t cf-ips "loaded $count Cloudflare prefixes"
```

Create `/etc/systemd/system/cf-ips.service`:

```
[Unit]
Description=Load Cloudflare IPv4 ranges into nftables set cf4
After=nftables.service network-online.target
Wants=network-online.target
PartOf=nftables.service

[Service]
Type=oneshot
ExecStart=/usr/local/sbin/update-cf-ips

[Install]
WantedBy=multi-user.target nftables.service
```

Create `/etc/systemd/system/cf-ips.timer`:

```
[Unit]
Description=Refresh Cloudflare ranges daily

[Timer]
OnCalendar=daily
RandomizedDelaySec=1h
Persistent=true

[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now cf-ips.service cf-ips.timer
sudo nft list set inet filter cf4     # expect around 15 prefixes
```

The line is IPv4-only behind the ISP router, so no IPv6 set is needed. Add one from `ips-v6` if IPv6 is ever routed to the lab.

12. Verification and acceptance tests

All tests must pass, or have a documented deviation, before further sites are published. External tests use a device off the home network, such as a phone on mobile data.

ID	Test	How	Expected
P01	Caddy listeners	`sudo ss -lntp` on the proxy	10.0.40.10:443, 10.0.40.10:8443, 127.0.0.1:2019 only
P02	Website loads publicly	Browse https://www.spaldotechoperations.co.uk externally	Page loads; edge certificate from Cloudflare
P03	Apex redirects	`curl -sI https://spaldotechoperations.co.uk`	301 to https://www…
P04	Security headers	`curl -sI https://www.spaldotechoperations.co.uk`	HSTS, nosniff, X-Frame-Options present; no Server or X-Powered-By from the origin
P05	Real client IP logged	Request externally, then `sudo tail -n1 /var/log/caddy/access.log`	`client_ip` is the test device's public IP, not a Cloudflare address

ID	Test	How	Expected
P06	Cloudflare cannot be bypassed	Externally: `curl -m5 --resolve www.spaldotechoperations.co.uk:443:<home IP> https://www.spaldotechoperations.co.uk`	Times out
P07	Cloudflare set loaded	`sudo nft list set inet filter cf4`, and CF_IPV4 in OPNsense diagnostics	Both populated
P08	Plex works remotely	Plex app on mobile data, or https://plex.spaldotechoperations.co.uk:8443/web	Connects with a valid Let's Encrypt certificate; not via Relay
P09	Game server reachable	Connect a client to play.spaldotechoperations.co.uk	Joins
P10	Only registered ports open	External scan of the home IP	Exactly the ports in §8.1; 32400, 80 and management ports closed
P11	Register matches devices	Compare §8.1 with ISP router forwards and OPNsense NAT rules	Identical, rule names carry F-numbers; UPnP off
P12	Dynamic DNS current	`dig +short home.spaldotechoperations.co.uk @1.1.1.1` vs https://ifconfig.me	Same address
P13	DMZ cannot reach Management	From proxy: `nc -zvw3 10.0.10.1 443`, `nc -zvw3 10.0.10.10 53`	Both time out
P14	DMZ cannot reach Home or Proxmox	From proxy: `nc -zvw3 192.168.68.10 8006`, `nc -zvw3 192.168.68.1 80`	Both time out
P15	DMZ reaches Plex on 32400 only	From proxy: `nc -zvw3 <PLEX_IP> 32400`, then `nc -zvw3 <PLEX_IP> 22`	First succeeds, second times out
P16	Only approved DNS	From proxy: `dig @8.8.8.8 example.com +time=3`	Times out; `dig @1.1.1.1` succeeds

ID	Test	How	Expected
P17	SSH from admin source	`ssh sysadmin@10.0.40.10` from MGMT or the admin workstation	Key login succeeds
P18	SSH from other sources	Same command from a non-admin home device	Times out
P19	SSH password rejected	`ssh -o PubkeyAuthentication=no sysadmin@10.0.40.10`	`Permission denied (publickey)`
P20	Web backends not reachable directly	From a home client: `curl -m5 http://10.0.40.20:8080`	Times out
P21	Plex from home via split DNS	From a home client: `nslookup plex.spaldotechoperations.co.uk`, then open it on :8443	Resolves to 10.0.40.10; loads with a valid certificate
P22	Survives reboot	`sudo reboot`, then repeat P02, P07, P08	All pass within 2 minutes
P23	Backup exists	Proxmox → Backups for VM 140	bkp-core-proxy-daily has a completed run

Record results alongside SPO-HL-SEC-001 tests H01–H16. Re-run P06, P10 and P11 after any change to the register.

13. Backup, operations and sign-off

13.1 Backup

- **VM backup:** Proxmox job `bkp-core-proxy-daily` to PBS, retaining 7 daily and 4 weekly copies (Technical Implementation Guide §11.2, DMZ guests). PBS encryption must be on, because the VM holds the Cloudflare API token.

- **Config archive:** after every change, take an encrypted copy of the proxy's configuration and store it off the host. Export the OPNsense configuration at the same time, since it holds the forwards and the DDNS token:

```
sudo tar czf - /etc/caddy /etc/nftables.conf /etc/ssh/sshd_config.d \
  /usr/local/sbin/update-cf-ips /etc/systemd/system/cf-ips.* \
  | gpg --symmetric --cipher-algo AES256 -o core-proxy-01-config-$(date
+%F).tar.gz.gpg
```

- **Restore test:** restore VM 140 to a throwaway ID with its NIC disconnected (`qm set <id> --net0 virtio,bridge=vmbr0,tag=40,link_down=1`). Otherwise it collides with the live 10.0.40.10. Check the services start, then destroy it.

13.2 Publishing a new service

1. Decide its path. A website goes via Cloudflare on 443 and needs no new forward. Another HTTP service that cannot use Cloudflare goes on Caddy's 8443 and needs no new forward either. Anything that is not HTTP gets its own forward row.
2. Add the register row in §8.1 first, with the next F-number, if a new forward is needed.
3. Create the DNS record in Cloudflare: a proxied CNAME to the apex for websites, or a DNS-only CNAME to `home` for everything else.
4. For HTTP services, add a Caddy site block (§7), then `sudo caddy validate` and `sudo systemctl reload caddy` .
5. For new forwards, add the ISP router forward and the OPNsense port forward with its associated rule, both named with the F-number.
6. Add any new backend to OPNsense rules and nftables egress, and restrict the backend's own firewall to its single upstream.
7. Run P02–P06 (websites), P08–P11 (direct services), and update this document.

13.3 Operations

Frequency	Activity
Weekly	Review WAF events in Cloudflare and Suricata alerts on WAN; `systemctl status caddy cf-ips.timer`
Monthly	`sudo caddy upgrade` then restart; review access logs and `nft-` drops; confirm DDNS records match the public IP

Frequency	Activity
Quarterly	Isolated restore test (13.1); walk the register against the ISP router and OPNsense (P10, P11); remove forwards for retired AMP instances
Before token expiry	Rotate the Caddy and DDNS Cloudflare tokens separately, update `caddy.env` and the OPNsense account, revoke the old tokens
After every change	Config archive and OPNsense export, P01–P23, change record per SPO-HL-SEC-001 §11

13.4 Sign-off checklist

- [ ] VM built per §4 with guest firewall enabled and a clean-install snapshot.
- [ ] Debian hardened: HTTPS apt, unattended upgrades, key-only SSH, no root login.
- [ ] Custom Caddy with the Cloudflare DNS and IP-range modules installed via dpkg-divert.
- [ ] Caddy API token scoped to one zone with Zone Read and DNS Edit only, stored at mode 0640.
- [ ] Caddy listening on 10.0.40.10:443 and :8443 only, production certificates issued.
- [ ] Forward register in §8.1 complete; ISP router and OPNsense match it; UPnP off on the ISP router.
- [ ] F1 restricted to CF_IPV4 at OPNsense and to `cf4` in nftables, with both lists populated.
- [ ] Plex remote access off, custom URL set, 10.0.40.0/24 not in LAN Networks.
- [ ] Cloudflare records and zone settings applied per §9; dynamic DNS updating both records.
- [ ] Pi-hole split-DNS records for plex and play in place.
- [ ] OPNsense DMZ and inbound rules per §10; NAT reflection off.
- [ ] nftables and cf-ips enabled per §11.
- [ ] Tests P01–P23 passed or deviations recorded.
- [ ] Backup job running; first config archive and OPNsense export stored off the host.
- [ ] SPO-HL-SEC-001 tables updated to show VLAN 40 as the DMZ.

Sources

- caddy-dns/cloudflare module
- caddy-cloudflare-ip module

- Caddy: build from source and Debian package support files
- OPNsense: aliases, including URL Tables
- Cloudflare IPv4 ranges